VU AI HUB · LECTURE NOTES

Knowledge and Data

Year 2, September to October 2026 (in progress)

Written by Inaaya and shared on Notion. Original page: cheddar-wall-f3b.notion.site/3c09801a9309801a8df2f479c0c5322d

Contents

1. 31st of August
2. 2nd of September
3. 7th of September
4. 9th of September
5. 14h of September
6. 19th of September
7. 21st of September
8. 23rd of September
9. 28th of September
10. 2nd of October

31st of August

- Data: raw facts with no context (ex. “8.0, 7.5, 5.5, 4.0”)
- Information: data + context (ex. “these are grades for the Knowledge and Data course”)
- Knowledge: information + rules/experience that let you actually use it (ex. “based on these grades, I can tell if my study method is working”)

Knowledge

- Tacit knowledge (aka implicit knowledge) is the knowledge that a person retains in their mind
- Explicit knowledge (aka formal knowledge) is knowledge that has been formalized, codified and stored

Predictable inference is the goal, it’s a two way process where we want people to look at something in the same way with the same knowledge

- Formal knowledge can help to interpret and reuse data and make it reusable for other purposes

Knowledge graphs

A useful way of formally representing data, information and knowledge as a network of nodes (things) connected by labeled arrows (relationships)...

- that are heterogeneous
- in such a way that others (systems/people) can interpret a piece of data correctly
- by making the semantics/meaning of a piece of information explicit
- using graph (network)
- explicitly on the Web

By using these graphs you can make the knowledge explicit

Logic

A formal language with a syntax (which expressions are legal), semantics (what legal expressions mean, the meaning of each sentence with respect to interpretations and which expressions are entailed by other expressions) and calculus (how to determine meaning for legal expressions).

Syntax

A well-formed sentence is built by the following rules:

- 2 terms with a comparator between them
- a term is either a natura number, a variable, or a complex term
- a complex tern is an operator +, -, *, / applied to two terms ...

Semantics

Truth is defined in terms of assignment for variables

- $x + 2 < x + 3$ is true for all possible assignments, for all values
- $x + 2 \geq y$ is false when $x = 0$ and $y = 7$

Entailment: predictable inference

- from the sentence $x + y < 6$ we can conclude that $x < 10$

  → does $x + y < 6$ entail $x < 10$?

Knowledge Graph Logic

Syntax

A 'thing' can be two things:

1. Vocabulary (V) - no distinction between objects and literals
2. Predicates (P) - relations

Semantics

A KG is a set of triples (subject, predicate, object)

- We need to assign values to those triples their intended meaning, which was ? the mathematic object of a graph

Triple is true if the mapping on the left, right and the relation is true,

- an interpretation is model of a knowledge base if it is a model of all triples
- a triple is entailed by a knowledge base (KB) if it is true in all models of KB

2nd of September

Propositional logic

Syntax

P_1 and P_2 can be used as sentences, by using the next symbols in between them

- Negation $\neg$
- Conjunction $\wedge$
- Disjunction $\vee$
- Implication $\Rightarrow$
- Biconditional $\Leftrightarrow$

Semantics

A sentence can either be true or false
Complex statements are compositions of simple statements. Their truth value is calculated from the meaning of its parts

- Conjunction ∧ *and*
  - only true when both sides are true
- Disjunction ∨ *or*
  - true when at least one side is true
- Implication ⇒
  - true when both sides are true or both sides are false
  - also true when the right side is true
- Double implication ↔
  - only true when both sides are true or both sides are false
- Semantically equivalent ≡
  - only true when both truth tables have the same values in the same order
    - `p → q` and `¬p ∨ q`

Tautology

- A formula is a tautology if it's TRUE in every single row of its truth table
  - `p ∨ ¬p`
- A contradiction is the opposite, it's FALSE in every row

Semantic Entailment

- Formula A is entailed by formula B if whenever B is TRUE, A is also TRUE
  - The truth tables don't have to be exactly the same, for example when B is FALSE, A can be TRUE, as long as whenever B is TRUE, A is also TRUE
- If entailment doesn't hold that means that you found at least one row where B is TRUE and A is FALSE
  - That row is called a counterexample

Knowledge Base

- A knowledge base is a set of sentences that we assume to be true, written in curly bracket notation
- Models are interpretation functions of the variables, or validations, which evaluate all formulas in a KB to true
  - In other words, truth values that make a knowledge base true

---

Concept Hierarchies

- A concept is a category/idea (like Lions, Animals, Mammals)

Syntax

- You write axioms like `(Lions subclassOf Mammals)`, which means that Lions is a type of Mammal. A bunch of these axioms together = a 'knowledge base'

Semantics

- If you have a universe U of objects and each concept gets assigned a subset of that universe (ex. "Lions" = some specific objects)
- An axiom `(c1 subclassOf c2)` is TRUE if every object assigned to c1 is also assigned to c2 (subset relationship)
- If an assignment makes all axioms in the knowledge base TRUE, it's called a model

---

Knowledge Graphs on the Web

Tim Berners-Lee (inventor of the Web) wanted a "Web of Data", all your data is now trapped in their respective apps and they don't communicate with each other. There is a "Web of Documents" not one of

data. His idea was a "semantic web" where data from different sources are linked together, like how web pages are linked today. He had to solve two problems:

1. Semantic Integration - making data from different sources use the same/compatible meanings (so "Person" in one dataset matches "Person" in another)
2. Physical Integration - connecting data that's stored in totally different, scattered places

The solution was Linked Data

4 Proposals for Linked Data

1. Give everything a unique name: everything you want to talk about (a person, a book, a place) gets its own unique identifier, like how everyone labeled things in real life so there's no confusion about what's being referred to
2. Names are web addresses (URIs): instead of just "Rembrandt", you use something like `http://data.rijksmuseum.nl/person/Rembrandt` as the name, this works because:
   - It's globally unique (no two different thigs will accidentally have the same web address)
   - Since it's a URL, you can actually visit it and get more info about that thing
   - Different organizations can host their own data on their own servers, but everything's still connected through these addresses (like a map showing data hosted in different countries, all linked together)
3. Relations connect things into a graph: facts are written as simple `(subject, relation, object)` triples, like `(claire_danes, starred_in, the_hours)`. When you connect lots of these triples together, you get a Knowledge Graph, a big web of connected facts
4. Make the meaning (semantics) explicit and formal:
   - Don't just store data, also store the model behind it (what type of thing each item is, what type of thing each relationship is, and how types relate to each other in a hierarchy)
   - Add rules so computers can calculate/infer new facts, and rules that rule out wrong interpretations
   - Key idea: **"Semantics = predictable inference"**, meaning that if I know the rules, I can predict exactly what you'll conclude when I give you new information (even in a made-up language you don't understand)

Relationship to AI

- Knowledge graphs fall under "Knowledge Representation and Reasoning", one of the main branches of AI (alongside Machine Learning, Neural Networks, Robotics, Computer Vision, etc.)
- There are two big "families" of AI:
  - Subsymbolic AI = Neural Networks, Deep Learning (pattern-based, learns from data)
  - Symbolic AI = Knowledge Representation and Reasoning (rule-based, uses explicit logic, this is what the course is about)
- Neurosymbolic AI combines both: neural networks handle "learning" (recognizing patterns) while knowledge graphs handle "reasoning" (logically explaining why something is true)

7th of September

Linked Data & Semantic Web

4 proposals for Linked Data:

1. Give all things a name
2. Make those names addresses on the Web (so you can look them up)
3. Let relations between things form graphs of data

4. Add explicit semantics (formal meaning) so computers can reason predictably

Points 1-3 combined = a global graph of Linked Data

Querying on the web (1-3) (Linked Data)

- We need data to be connected
- Accessible on the web
- Represented in known data format

Inferencing on the Web (4) (Semantic Web)

- We need a shared model
- Defined formal semantics
- Predictable inferencing

Linked Data → add formal semantics → Semantic Web

Resource Description Framework (RDF)

Triples

All information in RDF is expressed as triples: subject - predicate - object:

Subject	Predicate	Object
The Netherlands	has capital	Amsterdam
Amsterdam	has mayor	Femke Halsema
Femke Halsema	birth year	1966

Another word for a triple is a statement or a fact or an axiom

The elements of an RDF triple are either URI references, blank nodes, or, literals:

- Subject: URI or blank node
- Predicate: URI only
- Object: URI, blank node, or literal

Uniform Resource Identifiers (URI's)

A **URI** (Uniform Resource Identifier) is the broader concept, a string that identifies a resource, whether that resource is a real thing on the web or just an abstract name for something (a concept, a person, a relationship type)
A **URL** (Uniform Resource Locator) is a specific *kind* of URI, one that not only identifies a resource but also tells you where to find it and how to get it (the protocol, e.g. `http://`, plus a location, e.g. a domain and path)
A **URN** (Uniform Resource Name) is the other main kind of URI, one that names a resource persistently and uniquely, without saying anything about where to find it or how to retrieve it. It's pure identification, no location

So: every URL is a URI, but not every URI is a URL

Resources are identified by URI's, or URI's denote resources

> (URI's can only refer to a resource, they are not a resource, and multiple URI's can denote the same resource)

Internationalized Resource Identifiers (IRI's) are URI's that allow unicode characters

URI's are not URL's

- URI's are often long and difficult to read and write

- Most serializations use an abbreviation mechanism: namespaces and prefixes

Why HTTP?

- HTTP URI's have a global scope, unique throughout the Web
  - Helps to avoid name clashes
  - Grounded in society - people know how to use them
- HTTP URI's are also addresses
  - Exploit the well-functioning machinery in Web Browsing
  - Track data by following the resource identifiers found in triples

Literals

- Literals are used to represent 'literal' data values
- All literals have a datatype
- Literals can be annotated with a datatype
- Default: if no datatype is specified, the datatype is assumed to be `xsd:string`
- Datatypes are also resources and referenced via URI's
- One can also specify the language of a string using a language tag

Graphs

- An RDF graph is a set of triples: often drawn as a directed labeled graph (nodes = subjects/objects, arrows = predicates)

Why triples?

- We used triples because every piece of information can be transformed in this format
- Relationships are made explicit; they are elements in their own right
  - Unlike database columns and binary predicates
  - The predicate is an element in the triple, and can be described in RDF
  - 'Self-documenting'

Why graphs?

- A single, but highly versatile format
  - Everything is the same format: triples
- Since RDF graphs are just sets of triples, basic set operations are well-defined
- Merging two RDF graphs? Take their union
  - With tabular data, table dimensions must match
  - With trees, a node can only have one parent
- Extending an RDF graph? Add more triples
  - No need to redefine the database table, or to restructure the XML schema

Problems

- Can all information be represented as triples of URI's and Literals?

Blank Nodes

- Blank nodes are resources without a URI
- Use them when resource is unknown, or has no (natural) identifier

Triple Grammar

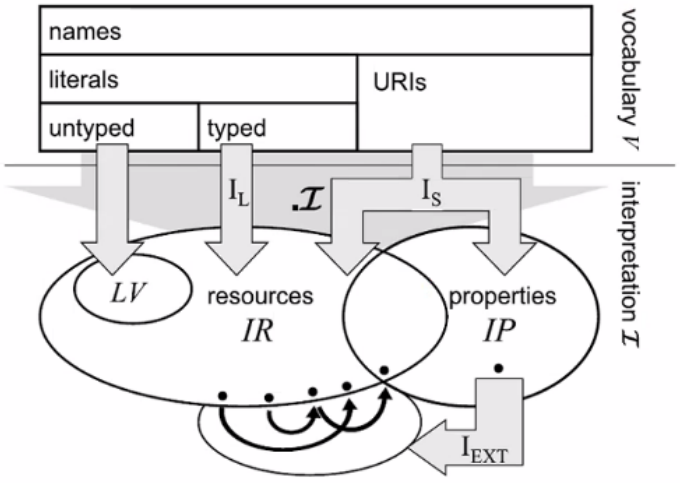
- Literals and Blank Nodes may not appear in every position of a triple
- literals are just values, no relationships from literals allowed
- blank nodes in predicate position 'too meaningless' and confusing

---

RDF Syntaxes (how RDF is written down)

- RDF/XML - the original, but outdated and hard to read
- N-triples - one full, unabbreviated triple per line, ending in a period; easy for machines to parse. URIs go in `< >`, literals in `" "`, blank nodes start with `_:`
- Turtle - a more convenient, human-friendly syntax:
  - Comments start with `#`
  - Statements (triples) end with a period
  - `a` is shorthand for `rdf:type`
  - `@prefix` declares namespace abbreviations (can even set a default, unnamed prefix)
  - Literals go in quotes; numbers/booleans can be written without quotes
  - Shorthand: if several triples share the same subject, separate them with `;`. If they also share the same predicate, separate values with `,`
  - Blank nodes can be written as `_:name` or inline using square brackets `[ … ]`

9th of September



Reusing Things: Vocabularies

- Since URIs are globally unique, we can reuse the same terminology (predicates, classes) across completely different datasets instead of inventing new ones every time
- This reuse is necessary for:
  - Data integration - different datasets can "speak the same language"
  - Inferencing - reasoning needs consistent, shared definitions
- A vocabulary (also called an ontology or schema) defines:
  - Properties to use as predicates
  - Classes to assign types to resources
- Think of vocabularies like software libraries, but for data - don't reinvent the wheel, reuse an existing one when possible (ex. `foaf`, `dcterms`, `skos`, `schema.org`)
- Because names are also web addresses (point 2 of the 4 Linked Data proposals), the exact same URI for a class like `Person` can be used by completely different people/organizations, in different locations, and everyone's statements still connect through that shared URI
- Important consequence: because anyone can publish RDF about anything using shared vocabularies, "the web is social" (anyone can say anything about anything). Ex. DBpedia, IMDB and Rotten Tomatoes

can all make independent (and even contradictory) statements about the same film using the same subject URI.

RDF Semantics

- "What does an RDF graph actually mean"
- An interpretation is a mathematical structure that assigns real-world meaning to the names in a graph: it maps names (literals, URIs) to actual resources and properties
- An interpretation is a model of a triple if the subject and object map to things actually related by the predicate's property
- An interpretation is a model of a whole graph if it makes every triple in that graph true (satisfies every axiom the graph)
- Entailment: a graph G entails another graph G' if every interpretation that is a model of G is automatically also a model of G'. Practically: G' doesn't tell you anything new/extra that wasn't already implied by G, it's safely derivable from G. Ex. a bigger graph entails all of its sub-triples
- Blank nodes complicate this slightly: since a blank node isn't a specific resource, an interpretation only needs to satisfy the graph for at least one possible assignment of blank nodes to actual resources (this is why blank nodes behave like 'there exists some X' in logic)
- This formal machinery is what lets reasoners later draw new true conclusions from existing RDF data in a predictable, checkable way (this is also the theoretical foundation that RDFS/OWL reasoning builds on)

SPARQL

> *SPARQL Protocol and RDF Query Language*

- It has two separate standardized parts:
  - The SPARQL Protocol - defines how to send queries to a triple store over HTTP
  - The SPARQL Query Language - defines the actual query syntax you write
- A SPARQL endpoint is a web service (a URL) that accepts a SPARQL query and returns results, it's the standard HTTP REST API for querying a triple store

The basic structure of a SPARQL query

1. Prologue (optional) - `PREFIX` declarations, similar to Turtle's `@prefix` but written slightly different (no trailing `.`)
2. Query clause - declares what kind of answer you want back. This is the 'verb' of the query, four main froms:
   - `SELECT`: Graph → Table. Returns a table of variable bindings. The most commonly used form
   - `ASK`: Graph → true/false. Just answers a yes/no question
   - `CONSTRUCT`: Graph → Graph. Builds and returns a brand-new custom RDF graph from the matches
   - `DESCRIBE`: URI(s) → Graph. Returns "everything known" about a resource (every triple where it's the subject or object)
   - (`INSERT` behaves like `CONSTRUCT` but actually writes the resulting graph into the triple store instead of just returning it, used for updating data, not just querying)
3. WHERE clause - the heart of the query: a graph pattern describing the shape of data you're looking for, with some parts replaced by variables
4. Modifiers (optional) - things like `ORDER BY`, `LIMIT`, `DISTINCT`, `GROUP BY` that shape/filter/sort the final results

Graph patterns & triple patterns

- The WHERE clause's graph pattern is essentially an RDF graph where some nodes/edges are variables (written with a `?`, ex. `?x`, `?y`)
- It's built from one or more triple patterns, a triple pattern is just a triple (subject, predicate, object) where zero or more parts are replaced by variables, ex:

```
?x dbo:capital dbpedia:Amsterdam .
?x dbo:capital ?y .
?x dbo:areaCode "020" .
?x ?p ?y .
```

- The query engine goes through the data and finds every pair that fits
- A pattern can match multiple times in the data, producing multiple result rows, each distinct match = one row
  - If the pattern fits 10 times in the data, you get 10 rows

Combining patterns: conjunction, disjunction, optional

- Conjunction (AND) - by default, multiple triple patterns in one `WHERE` clause must all match simultaneously (using the same variable bindings across patterns), ex:

  ```
  SELECT ?x WHERE {
  ?x dbo:capital ?y .
  ?y dbo:areaCode "020" .
  }
  ```

  This only returns `?x` values where both lines are true together, it won't match 'capital of X has code 020' with a different, unrelated city that also happens to have that code
- Disjunction (OR) - use `UNION` to say "at least one of these patterns should match":

  ```
  SELECT ?y WHERE {
  { ?y dbo:areaCode "020". }
  UNION
  { ?y dbo:areaCode "010". }
  }
  ```
- `OPTIONAL` - marks part of the pattern as not required to match; if it doesn't match, that part just comes back empty/unbound instead of excluding the row entirely:

  ```
  SELECT ?x ?y WHERE {
  ?y dbo:areaCode "020" .
  OPTIONAL { ?x dbo:capital ?y . }
  }
  ```

  So you still get `?y` even if no `?x` happens to have it as a capital

Filtering results

- `FILTER` adds a condition that must be true for the already-matched bindings to be kept, it doesn't match new triples, it just tests/validates values from a successful match:

  ```
  SELECT ?y WHERE {
  ?y dbo:areaCode "020" .
  ?y dbo:populationTotal ?x .
  FILTER (?x > 500000)
  }
  ```

Solution modifiers

- `ORDER BY` - sort results (`ORDER BY DESC(?label)` for descending)

- `DISTINCT` - remove duplicate rows from the result table
- `LIMIT` - cap the number of results returned (ex. `LIMIT 10`)
- These can all be combined in one query

More advanced features

- Named Graphs - `GRAPH <uri> { … }` lets you scope a pattern to a specific named sub-graph (context) instead of the whole dataset
- Aggregates - `COUNT`, `SUM`, `MIN`, `MAX`, `AVG`, `GROUP_CONCAT`, `SAMPLE` - compute summary values over matches
- Grouping - `GROUP BY` (group rows before aggregating) and `HAVING` (filter groups, like `FILTER` but for grouped results)
- `VALUES` - lets you inject a fixed set of data directly into the query (inline data) instead of only matching against the store:

```
SELECT ?country ?population WHERE {
VALUES ?country { dbr:France dbr:Germany dbr:Italy }
?country dbo:populationTotal ?population .
}
```

- Negation - `MINUS` excludes results that match a given pattern
- Functions - a large built-in function library, including:
  - Logic/testing: `IF`, `BOUND`, `EXISTS`, `NOT EXISTS`, `IN`, `NOT IN`, `&&`, `||`, `=`, `<`, `>`, etc.
  - Type checks: `isIRI`, `isBlank`, `isLiteral`, `isNumeric`
  - Value inspection: `str`, `lang`, `datatype`
  - String functions: `STRLEN`, `SUBSTR`, `CONTAINS`, `CONCAT`, `REGEX`
  - Date/time: `now`, `year`, `month`, `day`, `hours`, `minutes`, `seconds`

Federated queries

The `SERVICE` keyword lets one SPARQL query reach out to a different, external SPARQL endpoint mid-query and combine its results with the local data, ex. querying Wikidata end WikiPathways together in a single federated query. This lets you join data living in entirely separate triple stores without manually downloading and merging anything yourself

Practical workflow (e.g. with DBpedia)

- A resource page like `https://dbpedia.org/resource/Rembrandt` is human-browsable in a browser but also responds to content-negotiated requests for raw Turtle (`curl -H "Accept: text/turtle" ...`)
- DBpedia (and most triple stores) expose a dedicated SPARQL query editor/endpoint (e.g. `https://dbpedia.org/sparql`) where you paste in `PREFIX` + `SELECT ... WHERE { ... } LIMIT ...` and run it directly, or use a tool like Yasgui for a nicer interface with tables/charts
- DBpedia itself sits at the center of huge web of interlinked open datasets (the "Linked Open Data cloud") spanning publications, life sciences, government, geography, media, and more — all cross-referencing each other via shared URIs

14h of September

Publishing and consuming RDF

Where do the triples live?

- As separate files, ex. as .ttl, .rdf, .nt, etc. You download and load them, for example into RDFLib

- Inside web pages, using RDFa or Microdata
- Via content negotiation, you ask a URI for a specific format
  - `curl -L -H "Accept: text/turtle" https://dbpedia.org/resource/Amsterdam` gives you Turtle
  - The same URI with `Accept: text/html` gives you the normal webpage
  - in RDFLib you can do `g.load('http://dbpedia.org/resource/Amsterdam')`
- In triple stores, there are RDF-specific databases with a standard HTTP REST API for SPARQL, called a SPARQL endpoint

Dereferencing (getting data by following a URI)

- Simple case: the client sends GET to a URI and the server replies "200 OK" with RDF
- With content negotiation:
  - The client sends GET with `Accept: application/rdf+xml`
  - The server replies "303 See Other" and tells the client where the RDF actually lives
  - The client sends a second GET to that location and gets "200 OK" with the RDF
- `curl -L` means "follow the redirect"

Getting RDF out of a triple store

- Load files into a local triple store and query them with SPARQL
- From a program, either dereference a URI (`g.load(…)`) or use SPARQLWrapper
  - SPARQLWrapper sends a query to an endpoint, asks for JSON back, and you loop over `results["results"]["bindings"]`
  - This does not necessarily return RDF, it returns a table of values, not triples

Triple stores

- It is a purpose-built graph database for RDF
- It can store data on disk (more scalable) or in memory (faster)
- It usually comes with a GUI and a SPARQL interface
- Ex. GraphDB

Why they are fast:

- Dictionary: long names (URIs) are replaced by short numeric IDs, so the store compares numbers instead of strings
- Indexes: extra lookup tables, such as subject → triples, object → triples and property → triples, plus combinations like subject + property
  - For example, t1 = (s1 p1 o1), t2 = (s1 p2 o2), t3 = (s2 p1 o1). The subject index says "s1 → [t1, t2], s2 → [t3]"
- Statistics: the store uses them to plan queries
- This makes joins efficient
  - `?x a ex:Woman ; a ex:Professor`, where the store finds everything that is in both groups

---

Proposal for Linked Data (4): Make the meaning of things explicit

- You describe not just the data but also its underlying model
- You assign types to things, assign types to relations, and organize the types in a hierarchy
- You add rules for calculating with that knowledge. The rules put constraints on possible interpretations, and that is called entailment

Calculating with knowledge (inferencing/reasoning)

Inferencing is algorithmic manipulation of knowledge to derive new knowledge (where the meaning of words is not needed)

- All A is B, all B is C, therefore all A is C

The key point is that the algorithm doesn't need to know what the words mean. It only follows the form

Querying vs inferencing on the web

- Querying (P1-3) needs data that is connected, accessible on the web, and in a known format
  - Ex. with `Netherlands has_capital Amsterdam`, the query `?X has_capital ?Y` finds every matching pair directly
- Inferencing (P4) needs a shared model, defined formal semantics, and predictable inferencing
  - Example rule: if X is_a Y and Y subclass Z, then X is_a Z
  - Say the data says `Netherlands is_a Country` and `Country subClassOf GeographicalEntity`. Then the query `?X is_a GeographicalEntity` returns Netherlands, even though nobody wrote that triple
- Moving from querying to inferencing is the move from Linked Data to the Semantic Web

Knowledge representation: the principles

- A set of reserved symbols, like → for implication in logic, = for equality, and "isa" in semantic networks
- A set of variables than can be given values (p, q, ?x, ?y)
- A formal semantics that defines when formulas are true and how formulas relate to each other (entailment)
- A set of inference rules for manipulating formulas, such as Aristotle's syllogisms or modus ponens ("from p and (p → q), infer q")
- The rules are meaningful only if they follow the meaning defined in the semantics
- A set of formulas (axioms) about the world that the rules can be applied to, this is your domain knowledge

Two kinds of knowledge in a knowledge graph

1. Generic knowledge is also called the schema level, ontology level or T-box
   - It is represented by classes, a class is a set of individual things, like employee, person or car
2. Factual knowledge is also called the data level, instance level or A-box
   - It is represented by individuals (instances), an individual is one specific thing, like laurent, my car or this duck

- Relations between classes and individuals are called properties (or roles), a property is a pair of things, like `Laurent worksFor ensisa`
- In the diagram, the T-box has employee → person → workfor → company, and the A-box has laurent → workfor → ensisa

Model and reality

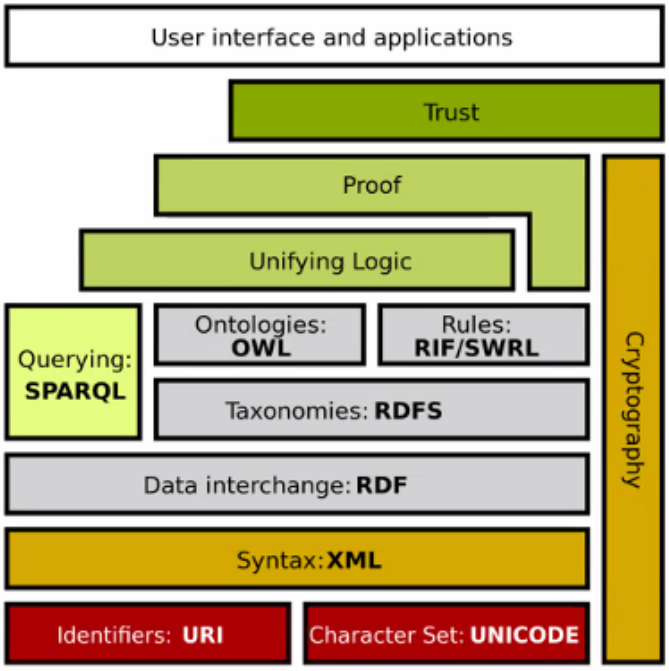
- Your knowledge base has symbols: `ex:Arch` is a class and `ex:Arc_de_Triomphe` is an individual, linked by rdf:type
- An interpretation says what those symbols point to in the real world
  - `ex:Arch` denotes the class of all arches
  - `ex:Arc_de_Triomphe` denotes the actual monument in Paris
- Classes can be overlapping, like Woman, Politician, Female, Politician, City, Country and House

What formulas can express

- Conditions on class membership, like "all mammals are warm-blooded"

- Relations between classes, like “all cities are populated places”
- Assertions of class membership, like “Amsterdam is a city”
- Characteristics of properties, like “hasCapital only relates countries to cities” or “partOf is transitive”
- Assertions of property relations, like hasCapital(Netherlands, Amsterdam)
- Assertions of equality, like morning star = evening star = Venus
- Formulas are axioms that restrict the possible interpretations of the world. The interpretations that survive are the models of the knowledge base
- Entailment means true in all of those restricted interpretations (the models)

RDFS (RDF Schema)



- It is a W3C standard vocabulary that extends basic RDF
- It gives you built-in reusable classes and properties, which you can think of as software libraries
- It lets you state basic schema facts about classes and properties
- It defines things like subclass and subproperty, which structure the data and enable inferencing
- Prefixes: `rdfs:` is `http://www.w3.org/2000/01/rdf-schema#` and `rdf:` is `http://www.w3.org/1999/02/22-rdf-syntax-ns#`

5 basic things you can say in RDFS

- “This is a class” `ex:Country rdf:type rdfs:Class .`
- “This is a member of a class” `ex:Netherlands rdf:type ex:EuropeanCountry .`
  - This is very similar to the first one, both use `rdf:type`
- “This class is a subclass of another” `ex:EuropeanCountry rdfs:subClassOf ex:Country .`
- “This property always relates members of specific classes”
  - `ex:containsCity rdfs:domain ex:Country .` means the subject is a Country
  - `ex:containsCity rdfs:range ex:city .` means the object is a City
  - This is like a restriction, but it is not a formal one. It’s really an entailment about the types of the subject and object. Real restrictions come in OWL
- “This property is a more specific version of another”
  - `ex:hasCapital rdfs:subPropertyOf ex:containsCity .`

- `ex:hasCapital rdfs:range ex:Capital .`

RDFS Entailment Rules

#	Rule	If you have...	...you can add
1	Everything is a resource	`s p o`	`s` is a Resource, `p` is a Property, `o` is a Resource (or a Literal)
2	Types are classes	`s rdf:type o`	`o a rdfs:Class`
3	Class is its own subclass	`s rdf:type rdfs:Class`	`s subClassOf s`
4	Property is its own subproperty	`p rdf:type rdf:Property`	`p subPropertyOf p`
5	Subclass chains	`s subClassOf y`, `y subClassOf z`	`s subClassOf z`
6	Subproperty chains	`p subPropertyOf q`, `q subPropertyOf r`	`p subPropertyOf r`
7	Subproperty inheritance	`s p o`, `p subPropertyOf q`	`s q o`
8	Domain	`s p o`, `p domain x`	`s rdf:type x`
9	Range	`s p o`, `p range x`	`o rdf:type x`
10	Membership goes up	`s rdf:type o`, `o subClassOf t`	`s rdf:type t`

1. Anything that shows up in a triple counts as a thing, and a predicate counts as a property
   - from `Netherlands hasCapital Amsterdam`, you get “hasCapital is a Property”
2. If something is used as the type of another thing, it must be a class
   - from `Netherlands a EuropeanCountry`, you get “EuropeanCountry is a Class”
3. A class always counts as part of itself, this one is obvious but the formal system needs it spelled out
   - “Country is a subclass of Country”
4. The same as rule 3, but for properties
   - “likes is a subproperty of likes”
5. If A is inside B and B is inside C, then A is inside C
   - Cat is a subclass of Mammal and Mammal is a subclass of Animal, so Cat is a subclass of Animal
6. The same idea as rule 5, but for properties
   - hasCapital is a subproperty of containsCity and containsCity is a subproperty of contains, so hasCapital is a subproperty of contains
7. If the specific relations holds between two things, the broader relation holds between them too
   - `Netherlands hasCapital Amsterdam` plus “hasCapital is a subproperty of containsCity” gives `Netherlands containsCity Amsterdam`
8. If a property has a domain, anything that uses that property as a subject gets that type
   - `Victor likes Truffle` plus “likes has domain SentientBeing” gives `Victor a SentientBeing`
9. If a property has a range, anything it points to gets that type
   - `Victor likes Truffle` plus “likes has range Food” gives `Truffle a Food`
10. If something is a Cat, and every Cat is a Mammal, then it is a Mammal
    - `Truffle a Cat` plus “Cat is a subclass of Mammal” gives `Truffle a Mammal`

Which rule do I use?

`Cat subClassOf Mammal`

`Mammal subClassOf Animal`

`Truffle a Cat`

`Truffle likes CatFood`

`hasCapital subPropertyOf containsCity`

`Netherlands hasCapital Amsterdam`

`likes domain SentientBeing`

`likes range Food`

- Subclass chain: rule 5 for the classes, then rule 10 for the things inside them
  - Look for classes stacked like A → B → C
  - Rule 5 joins the classes: `Cat subClassOf Mammal` and `Mammal subClassOf Animal` give `Cat subClassOf Animal`
  - Rule 10 takes: `Truffle a Cat` and `Cat subClassOf Mammal` give `Truffle a Mammal`
  - Rule 10 again: `Truffle a Mammal` and `Mammal subClassOf Animal` give `Truffle a Animal`
- Subproperty: rule 7
  - Look for a property that is declared a subproperty of another
  - `Netherlands hasCapital Amsterdam` and `hasCapital subPropertyOf containsCity`
  - That gives `Netherlands containsCity Amsterdam`
- Domain or range: rule 8 types the subject, rule 9 types the object
  - Look for a property that has a domain or range, and a triple that uses it
  - The triple is `Truffle likes CatFood`, with `likes domain SentientBeing` and `likes range Food`
  - Rule 8 types the subject (the left side): `Truffle a SentientBeing`
  - Rule 9 types the object (the right side): `CatFood a Food`
  - Domain is the thing doing the "liking", range is for the thing being "liked"
- The "everything is a resource/class" facts: rules 1 to 4
  - These apply to almost any triple
  - Rule 1: from `Truffle likes CatFood`, you get `Truffle a Resource`, `CatFood a Resource` and `likes a Property`
  - Rule 2: from `Truffle a Cat`, you get `Cat a Class`
  - Rule 3: from `Cat a Class`, you get `Cat subClassOf Cat`
  - Rule 4: from `likes a Property`, you get `likes subPropertyOf likes`

19th of September

RDFS

Extra RDFS terms (not used for inferencing, but useful)

- `rdfs:label` - a human-readable name, like `ex:Country rdfs:label "Land"@nl`
- `rdfs:comment` - a description, like `ex:Amsterdam rdfs:comment "Amsterdam has two universities"@en`
- `rdfs:seeAlso` - points to another resource, like `ex:Amsterdam rdfs:seeAlso dbpedia:Amsterdam`
- Classes for "things":
  - `rdfs:Resource` is everything
  - `rdf:Property` is the type of all properties

- `rdfs:Class` is the type of all classes
- `ex:City a rdfs:Class` (`a` is the same as `rdf:type`)

How RDFS behaves

- Properties are first-class citizens, they are not part of a class, they exist on their own, which also makes domain and range statements tricky
- No strict split between schema and data, something can be an instance and a class at the same time (`ex:AirbusA380 rdf:type ex:PlaneModel` and also `ex:AirbusA380 rdfs:subClassOf ex:AirbusPlane)`
- No negation, you can't say x is NOT y for example
- No equality, you can't say two things are the same
- You can use the reserved symbols inside your own statements
- RDFS is not very expressive, that's why OWL exists

RDFS describing itself

- The RDFS definition file is itself written in RDF
  - ex. rdfs:Class a rdfs:Class ("the class of classes), and rdfs:subClassOf is a Property with domain Class and range Class

Property	Domain (subject type)	Range (object type)
`rdf:type`	Resource	Class
`rdfs:domain`	Property	Class
`rdfs:range`	Property	Class
`rdfs:subPropertyOf`	Property	Property
`rdfs:subClassOf`	Class	Class
`rdfs:label`	Resource	Literal
`rdfs:comment`	Resource	Literal
`rdfs:seeAlso`	Resource	Resource

Resource

- A resource is any 'thing' you can talk about in RDF: a person, a book, a city, a class, even a property
- Everything in the graph is a resource, so it's the broadest label there is
- Example: `ex:Amsterdam` , `ex:Truffle` and `foaf:Person` are all resources

Property

- A property is a resource that is used as a relationship between two things, it's the middle part of a triple
- Example: in `Netherlands hasCapital Amsterdam` , `hasCapital` is the property
- Properties are like the verbs and the other things the nouns

Domain and range

- Domain is the type of thing that goes on the left (the subject) of a property
- Range is the type of thing that goes on the right (the object)
- Example: `hasCapital` has domain `Country` and range `City`

Other Vocabularies

Vocabulary	Prefix	What it's for	Key terms

RDF	`rdf`	Reserved terms for the data model	`rdf:type` (links a resource to its type), `rdf:Property` (the type of all properties)
RDFS	`rdfs`	Schema terms	`Class`, `subClassOf`, `domain`, `range`, `subPropertyOf`, `Resource`, `Literal`, `Datatype`, `label`, `comment`, `seeAlso`, `isDefinedBy`, plus collection terms like `member` and `Container`
FOAF	`foaf`	People and the relations between them	`foaf:Person`, `foaf:name`, `foaf:mbox` (email), `foaf:knows` Example: `vu:Rinke_Hoekstra rdf:type foaf:Person ; foaf:knows vu:Albert_Merono ; foaf:name "Rinke Hoekstra"`
Dublin Core	`dct`	Bibiolographic info about documents	`dct:creator`, `dct:created`, `dct:description`, `dct:title` Example: `vu:SWPrimer dct:creator vu:Rinke_Hoekstra ; dct:created "2012"^^xsd:date ; dct:title "A Semantic Web …"`
schema.org	`schema`	Describing things on web pages, for search engines	Example: `http://scherma.org/Movie`
DBpedia	`dbo`, `dbr`, `dbp`	Facts extracted from Wikipedia infoboxes	`dbo:` for ontology classes and properties, `dbr:` for the resources (the actual things), `dbp:` for raw properties. Example: `dbr:Vrije_Universiteit_Amsterdam rdf:type dbo:University`
Geonames		Locations	
Data Cube (QB)		Statistical data	
SKOS	`skos`	Concept hierarchies and thesauri	see below
OWL	`owl`	Formal constraints on class membership	

SKOS

- A `skos:Concept` is a 'subject' used to index things, an `owl:Class`, on the other hand, is set of things
- A concept can correspond to both an instance and a class
- A narrower concept can be of a different type than its broader concept
- A concept is an RDF resource and can have multiple labels:
  - `skos:prefLabel` is the preferred label
  - `skos:altLabel` is an alternative label
  - Labels can be in different languages, like `"animals"@en` and `"dieren"@nl`
- Concepts can be grouped with `skos:member` of a `skos:Collection`, or `skos:inScheme` of a `skos:ConceptScheme`
- Relations between concepts:
  - `skos:broader` means more general, and `skos:narrower` means more specific
    - ex. Impressionist → Artist → Art
  - `skos:related` is symmetric. If A is related to B, then B is related to A
- `skos:broader` is more general than `rdfs:subClassOf`, it can mean:
  - a subclass or type

    - a part-of or location relationship
    - topic implication (like 'cow milk' under 'cows')
- Example: in the Truffle chain, you could swap the last step for `ex:Animal skos:broader ex:NaturalWorld`. That's looser than `subClassOf`, so the strict RDFS inferences no longer follow from it

Formal vs Informal Semantics

- Formal semantics: RDF, RDFS and OWL have strict formally defined semantics, RDF is defined over graphs and RDFS/OWL over sets
- Informal ('social') semantics: FOAF and many others are defined only in text descriptions and by how people use them online

---

Statement about statements and named graphs

The problem

- Plain RDF says `ex:painting ex:title "The Night Watch"` and `ex:painting ex:createdBy ex:Rembrandt`
- RDF has no easy way to say things about a triple, so if different sources disagree (about who painted The Night Watch for example) there's a problem

Solution 1: Reification (standard RDF)

- You turn the triple into a things of its own, using the built-in predicates (`rdf: subject`, `rdf:predicate`, `rdf:object`)
    - Ex. a blank node `_:st1` with `rdf:subject Painting001 ; rdf:predicate createdBy ; rdf:object Rembrandt ; accordingTo Rijksmuseum`
- It's standard, but it makes the graph huge and hard to read

Solution 2: RDF-Star

- A triple can appear as the subject or object of another triple
- Syntax: `<<ex: painting ex:createdBy ex:Rembrandt>> ex:accordingTo ex:Rijksmuseum`
- It means the data is no longer a strict directed labeled graph, but a hypergraph

Solution 3: Named graphs

- Triples are grouped into sets, and each set can have its own URI
- The graph acts as context for the triples inside it
- A triple can be in multiple named graphs, this is the 'quad' model (subject, predicate, object, graph), and it is also a hypergraph

```
ex:myGraph {
   ex:painting ex:createdBy ex:Rembrandt .
   ex:painting ex:title "The Night Watch" .
}
ex:myGraph ex:accordingTo ex:Rijksmuseum .
```

- Most triple stores treat each loaded file as its own named graph

21st of September

Complex modelling and advanced inferencing in OWL

RDFS is not always enough with all its restriction, that's why we also use OWL

OWL Basics

- OWL is built on description logics, which are small, carefully chosen pieces of logic
- The aim is a language that is expressive enough to be useful, but still decidable so that a computer can always finish reasoning and give an answer, which is also why OWL limits what you can say
- OWL is written in RDF, so an OWL ontology is just triples, using the `owl:` vocabulary
- OWL treats classes as sets of things, so subclass means subset and disjoint means the sets don't overlap
- No Unique Name Assumption (UNA): two different names might refer to the same things, so you can't assume `:Bob` and `:Robert` are different people
- Open World Assumption (OWA): if something isn't stated, it's unknown, not false. This is different from a database, which used the Closed World Assumption (CWA): not in the data means false
- `owl:Thing` is the class of everything. `owl:Nothing` is the empty class, and if something ends up in it, that's a contradiction

Class axioms (ways to describe classes)

- `owl:equivalentClass` – two classes have exactly the same members
- `owl:disjointWith` – no individual can be in both classes at once, like Cat and Dog
- `owl:complementOf` – everything that is not in the class, such as NonSmoker for Smoker
- `owl:unionOf` – the class is made of all members of several classes combined, for example Parent is Mother or Father
- `owl:disjointUnionOf` – a union where the parts also don't overlap
- `owl:intersectionOf` – the class is only the things in all of the listed classes, such as Mother = Woman and Parent
- `owl:oneOf` – you list the members explicitly, such as the Continents are exactly Europe, Asia, and so on

Property types

- Object property: links a thing to a thing, such as `:hasAuthor`
- Datatype property: links a thing to a value like a number or string, such as `:hasAge`
- Annotation property: just a note, like `rdfs:comment`, and the reasoner ignores it
- Property characteristics that can be switched on:
  - Symmetric: if A is married to B, then B is married to A
  - Asymmetric: if A is parent of B, B can't be parent of A
  - Transitive: is A is ancestor of B and B is ancestor of C, then A is ancestor of C
  - Functional: each thing has at most one value, such as one birth place
  - Inverse functional: each value belongs to at most one thing, such as a passport number
  - Reflexive: everything is related to itself
  - Irreflexive: nothing is related to itself, such as 'is parent of'

Property axioms

- `owl:inverseOf` : `hasChild` is the inverse of `hasParent`
- `owl:equivalentProperty` – two properties mean the same thing
- `owl:propertyDisjointWith` – two properties can't both hold between the same pair
- `owl:propertyChainAxiom` – if A has parent B and B has brother C, then A has uncle C (chaining relations together)

Class restrictions

- These define a class by what its members must be like, they're written as an `owl:Restriction` with `owl:onProperty`

- `someValuesFrom` – "at least one value of this property is from this class"
  - Ex. `NobelPrizeWinner` is anyone with at least one `wonPrize` linking to `NobelPrize`

Same and different individuals

- `owl:sameAs` – two names refer to the same thing, everything true of one is true of the other
- `owl:differentFrom` – they're definitely two different things
- This is needed because of no UNA, by default OWL doesn't assume names are different
- You can also say a property does not hold between two individuals, using `owl:NegativePropertyAssertion`
  - ex. Amsterdam is not LocatedIn Brussels

OWL Entailment Rules

#	Rule	If you have...	...you can add
1	sameAs works both ways	`v owl:sameAs w`	`w owl:sameAs v`
2	sameAs chains	`u owl:sameAs v` , `v owl:sameAs w`	`u owl:sameAs w`
3	Same thing, same facts	`v owl:sameAs w` , `v p o`	`w p o`
4	Symmetric property	`p a owl:SymmetricProperty` , `v p w`	`w p v`
5	Transitive property	`p a owl:TransitiveProperty` , `u p v` , `v p w`	`u p w`
6	Equivalent classes	`v owl:equivalentClass w`	`v subClassOf w` , `w subClassOf v`
7	Equivalent properties	`p owl:equivalentProperty q`	`p subPropertyOf q` , `q subPropertyOf p`
8	Functional property	`p a owl:FunctionalProperty` , `x p v` , `x p w`	`v owl:sameAs w`
9	Inverse functional property	`p a owl:InverseFunctionalProperty` , `v p x` , `w p x`	`v owl:sameAs w`
10	Property chain	`r owl:propertyChainAxiom (p q)` , `v p w` , `w q x`	`v r x`

1. If A is the same as B, then B is the same as A
   - from `Bob owl:sameAs Robert`, you get `Robert owl:sameAs Bob`
2. sameAs chains: if A is the same as B and B is the same as C, then A is the same as C
   - `A sameAs B` plus `B sameAs C` gives `A sameAs C`
3. If two names are the same thing, every fact about one is also a fact about the other
   - `Bob sameAs Robert` plus `Bob livesIn Paris` gives `Robert livesIn Paris`
4. A symmetric property works in both directions
   - marriedTo is symmetric, so `Anna marriedTo Tom` gives `Tom marriedTo Anna`
5. A transitive property passes along a chain
   - ancestorOf is transitive, so `Ann ancestorOf Bea` plus `Bea ancestorOf Cy` gives `Ann ancestorOf Cy`
6. Equivalent classes are subclasses of each other, in both directions
   - `Human equivalentClass Person` gives `Human subClassOf Person` and `Person subClassOf Human`
7. The same idea as rule 6, but for properties

- `writtenBy equivalentProperty hasAuthor` gives each one as a subproperty of the other

8. A functional property only allows one value, so if there are two values, they must be the same thing
   - hasBirthPlace is functional, so `Anna hasBirthPlace Paris` plus `Anna hasBirthPlace CityOfLight` gives `Paris sameAs CityOfLight`
9. An inverse functional property means a value belongs to only one thing, so if two things share the value, they must be the same thing
   - hasPassportNumber is inverse functional, so `A hasPassportNumber "X1"` plus `B hasPassportNumber "X1"` gives `A sameAs B`
10. A property chain: following p and then q gives you r
    - hasUncle is the chain (hasParent, hasBrother), so `Lea hasParent Mia` plus `Mia hasBrother Joe` gives `Lea hasUncle Joe`

23rd of September

Advanced modelling and inferencing in OWL

Necessary: if you're in class R, you have condition C

- ex. humans have at least one kidney
- They use `rdfs:subClassOf`, which means you can infer:
  - subclass relations
  - property values of things you already know are in the class

Necessary and sufficient: if you're in R you have C, and if you have C you're in R

- ex. a `redThing` is exactly a things that has colour Red
- They use `owl:equivalentClass`, which means you can infer:
  - class equivalence
  - that an individual is in the class

Types of restriction

- Existential (`someValuesFrom`) - all members of the class have at least one value of property p from class B
  - ex. Nobel Prize winners have won a Nobel Prize
  - Necessary and sufficient: `NobelPrizeWinner` is equivalent to 'things that have won at least one Nobel Prize' (`won someValuesFrom NobelPrize`)
  - Necessary only: `NobelPrizeWinner` is a subclass of 'things nominated for a Nobel Prize (`has some nominatedFor NobelPrize`)
- Universal (`allValuesFrom`) - all values of property p must be from class B
  - ex. `NobelPrize` is a subclass of 'things that are only `wonBy` `NobelPrizeWinners`'
  - You can't use `rdfs:range` because that applies to the property elsewhere, this only applies to `NobelPrizes` so it's less strict
    - When using `rdfs:range` someone could've only won a `NobelPrize`, there is no possibility for winning any other prizes
    - The other way around gives a wrong inference, when someone who is a `NobelPrizeWinner` wins something else that prize will be seen as a `NobelPrize`
- Specific value (`hasValue`) - all members have this specific individual as a value
  - ex. Amsterdam residents live in Amsterdam
  - With the `equivalentClass` the inference also works the other way around (in Amsterdam live Amsterdam residents)

- Min cardinality - at least n values
  - ex. A company has at least one employee
- Max cardinality - at most n values
  - ex. A professor supervises at most 10 PhD students
- Exact cardinality - exactly n values
  - ex. You can only be married to one person
- hasSelf - related to yourself
  - ex. People like themselves

Punning

- OWL keeps three things strictly separate: classes (sets), individuals (specific things) and properties
- Punning lets one IRI have more than one meaning, because sometimes things are both a category and a specific thing, depending on where it appears
  - ex. class and individual, object property and datatype property, class and datatype
- The meanings don't interfere with each other, and the reasoner treats them as separate things that just happen to have the same name
- The position of the IRI in an axiom decides which meaning it has
  - In `ex:sam a ex:Eagle`, `ex:Eagle` is a class, because it's the type
  - In `ex:Eagle a ex:Species`, `ex:Eagle` is an individual, because it's the thing being typed\

Common Mistakes

- Over-commitment
  - Every statement in your ontology is a commitment about the domain
  - Over-commitment means saying something too strong
- Domain and Range
  - If you say `hasColor` has domain `Wine`, then anything with a colour becomes a `Wine`, a red Ferrari would be inferred to be a wine
    - say `Wine` is a subclass of 'things with at least one `hasColor` value from Color' (`someValuesFrom`)
- Universal restrictions as equivalence
  - With `A equivalentClass (p allValuesFrom B)`, you can't conclude things the way you'd expect, because of the open world

    ```
    ex:alice ex:eats ex:salad .
    ex:salad a ex:VegetarianFood .
    ```

    - The reasoner won't conclude `ex:alice a ex:vegetarian` because it doesn't know everything she might've ate

    ```
    ex:bob a ex:Vegetarian .
    ex:bob ex:eats ex:mystery .
    ```

    - The reasoner will conclude `ex:mystery a ex:VegetarianFood`
- Complement
  - `MaleCat complementOf FemaleCat` means everything that isn't a female cat is a male cat, including a table or a car
    - say `Cat disjointUnionOf (MaleCat FemaleCat)`

28th of September

Ontology engineering and alignment

An ontology is an agreed-on-description of a topic, written in a language computers can read (RDF, RDFS, OWL), it has four parts:

1. Explicit: written down formally
2. Shared: people agree on it and publish it openly
3. Conceptualization: the classes, instances and relations you chose
4. Context: it's made for one domain or task

Ontology = the schema, instances = the data

Three ways to start

- Top-down: begin with the big general classes, then go down to details
- Bottom-up: begin with the data you already have
- Middle-out: begin with the most important concepts, then go both more general and more specific

Competency Questions

- These are the questions the users would want answered by the ontology
- They set the scope of the ontology and help you check it at the end

Ontology Development

There are 9 steps, it's not a straight line, you go back and forth:

1. **Domain and scope:** decide the topic and the competency questions
2. **Reuse:** look for an existing ontology first, there's almost always something to start from
3. **List terms:** write down all the relevant words, nouns become classes and verbs become properties
4. **Taxonomy:** put classes in hierarchy
    - Test: if A is a subclass of B, every A must be a B
    - (Amsterdam is not a 'kind of' Netherlands, it's part of it, so that's not a subclass)
    - Avoid cycles, and put shared restrictions high up so subclasses inherit them
5. **Properties:** think about domain and range straight away, and check whether they hold everywhere
6. **Classes and their properties:** add restrictions, cardinalities, property types and disjointness
7. **Instances:** usually filled in automatically from databases, CSV files or text, not by hand
8. **Check for anomalies:** use the reasoner to find contradictions, like conflicting cardinalities or domains
9. **Evaluate** with your competency questions

Ontology Alignment (linking different ontologies)

There will always be many overlapping ontologies, the Semantic Web doesn't fix that but it lets you state how they relate to each other.

Alignment means mapping between ontologies, merging means joining them into one new ontology. The automatic techniques compare:

- **Labels:** spelling, synonyms, WordNet
- **Structure:** shared parents or children
- **Instances:** two classes are similar if their members are

---

Which OWL link to use

- Individual to individual – `owl:sameAs` or `owl:differentFrom`

- Class to class – `owl:equivalentClass`, `rdfs:subClassOf` or `owl:disjointWith`
- Individual to class – `rdf:type` or punning
- Property to property – `rdfs:subPropertyOf`, `owl:equivalentProperty` or `owl:disjointProperty`

SKOS mappings

- SKOS links connect two thesauri, so you can move between them
- (they don't say anything formal, unlike OWL links)
- ex. `skos:exactMatch` (the same concept) and `skos:broaderMatch` (the other concept is broader)

Checking alignments

- Every mapping has a confidence level, and bad ones happen (`bank exactMatch sofa`, `couch exactMatch crouch`)
- Check precision link by link, or compare with an expert's reference alignment (precision and recall)

2nd of October

Data Integration

A semantic web application has data access, data integration and the application itself. At the application end, data is published or presented

Building the ontology (classes and properties) is manual, filling it with instances is usually semi-automatic

Getting tables into RDF

- Standards: Direct Mapping and R2RML (for databases), CSVW (for CSV), and the Data Cube vocabulary (for the table itself)
- Tools: OpenRefine, Ontotext Refine (OntoRefine), LD Wizard, COW, your own Python script with RDFLib, or find/replace in Excel
- From XML or JSON, use YARRRML

Workflow with GraphDB

1. Build your ontology in Protégé and import it into GraphDB (it runs locally at `localhost:7200`)
2. Get a CSV (from Excel, Google Dataset Search or Kaggle)
3. Convert it with OntoRefine:
    - load the CSV and open the RDF mapping
    - define a namespace
    - drag columns to subject, predicate and object
    - things are IRIs, and plain values (literals) can only be objects
    - IRIs can be built from a column value, and you can preview the result
4. Reuse existing vocabulary, find popular classes and properties on LOV (Linked Open Vocabularies) and paste their URIs in
5. Save as RDF and import it, or generate a SPARQL INSERT
6. Query it with SPARQL

Linking to outside data (like DBpedia)

- First look at DBpedia to see what exists and how to link to it (for example by name)
- Federated query: the `SERVICE` keyword sends part of your query to another SPARQL endpoint
- CONSTRUCT: like SELECT, but the result is RDF triples instead of a table, you can save it as Turtle

- INSERT: adds triples to your own store, usually combined with `SERVICE`. `DELETE` also exists (okay fun fact)
  - ex. match your local artists to DBpedia artists by name
    - INSERT `owl:sameAs` between each pair
    - INSERT extra facts like `dbo:notableWork` and `dbo:deathPlace`
    - SELECT using `owl:sameAs` to combine your data and DBpedia's
- Alternatives: the YASGUI website (CONSTRUCT, download as Turtle, import) or your own Python script

---

Machine learning and ontologies

- ML can help build ontologies, fill them with instances, align them, and build knowledge graphs from text
- Learning from text: find entities (NER), extract terms and relations, then learn a hierarchy. Techniques include TF-IDF and patterns like 'X is a type of Y'
- Alignment with ML: compare labels, structure and instances, often with embeddings, and predict mappings with confidence scores
- Population: find entities and relations in text and turn them into triples, a human should still check the results
- LLMs: can extract entities and relations, suggest classes, properties and mapping, and generate RDF. They're fast and scale well, but can hallucinate and be inconsistent, so you need to validate

Wikidata

- A knowledge base that powers much of Wikipedia (the facts that don't depend on language)
- It has a SPARQL endpoint at query.wikidata.org
- It uses qualifiers, which add extra detail to a statement